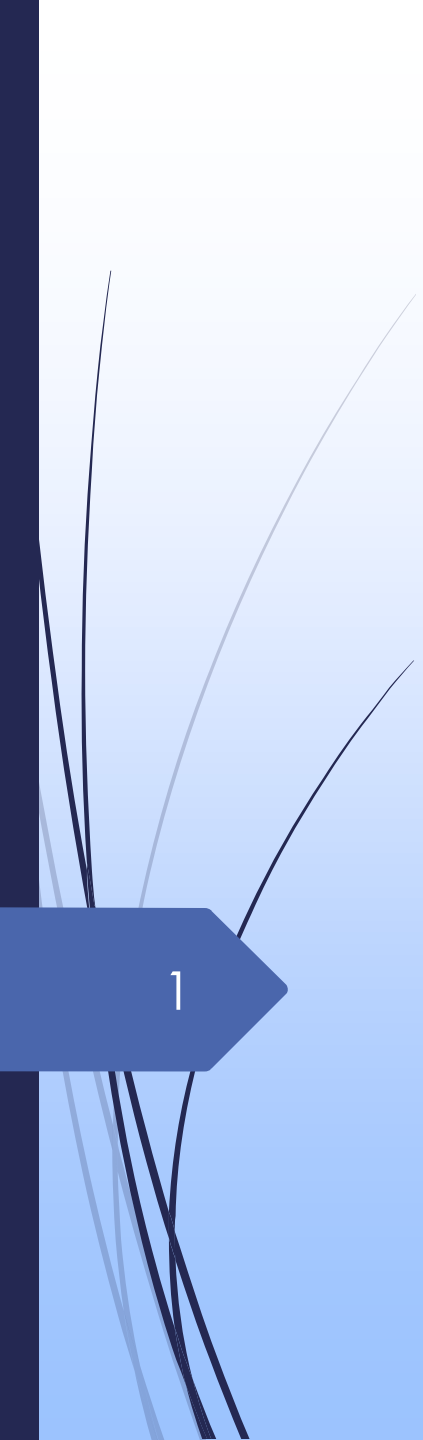




EECS 280 – Lecture 4

Machine Model, Part 1

1

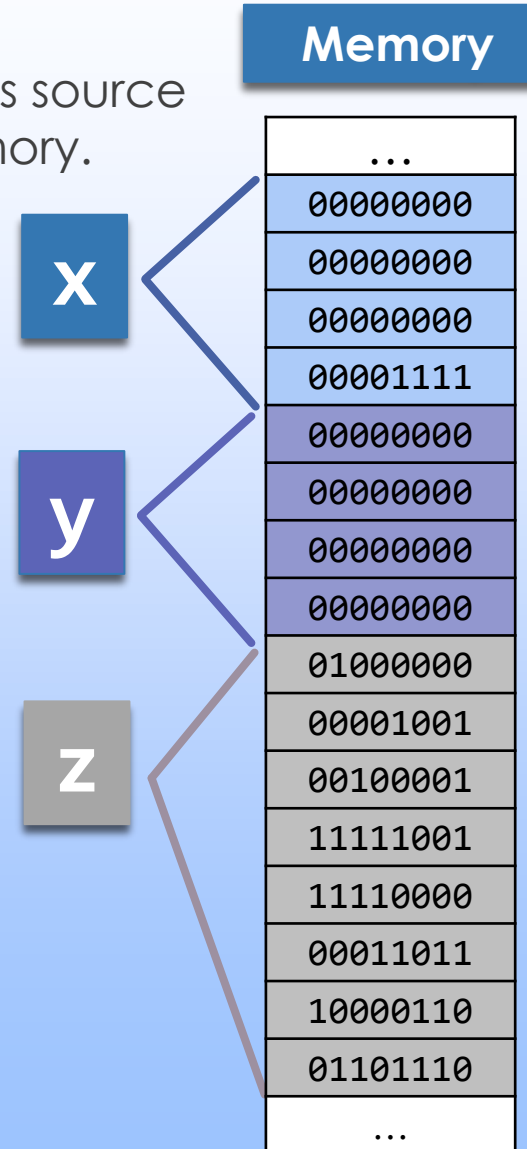
Recall: Variables and Memory

- A variable is a **name** in the program's source code that refers to an **object** in memory.

- **object** – region of memory holding data

```
// Declare some variables
int x = 10 + 5;
int y = x;
double z = 3.14159;
```

- Data is stored in **bytes**, composed of 8 **bits** - binary digits of 0 or 1.
- A variable's **type** determines:
 - How much space it needs
e.g. `int` = 4 bytes, `double` = 8 bytes
 - How its value is encoded to binary



Default Initialization

- Objects can be explicitly initialized:

```
int main() {  
    int x = 5;  
    double y = 7.8;  
    vector v = { 3, 4, 5 };  
    string s = "spaghetti";  
}
```

- Otherwise, they are **default-initialized**:

```
int main() {  
    int x;        // x = ??  
    double y;     // y = ??  
    vector v;     // v = { }  
    string s;     // s = ""  
}
```

- What does default-initialization do?

For atomic types like `int`, `double`, `bool`, `char`, etc.
“do nothing” – the value is undefined

For class types like `vector`/`string`, a **default constructor**¹ ensures the object has a meaningful default value.

¹ We'll also come back to this for custom class types we define...

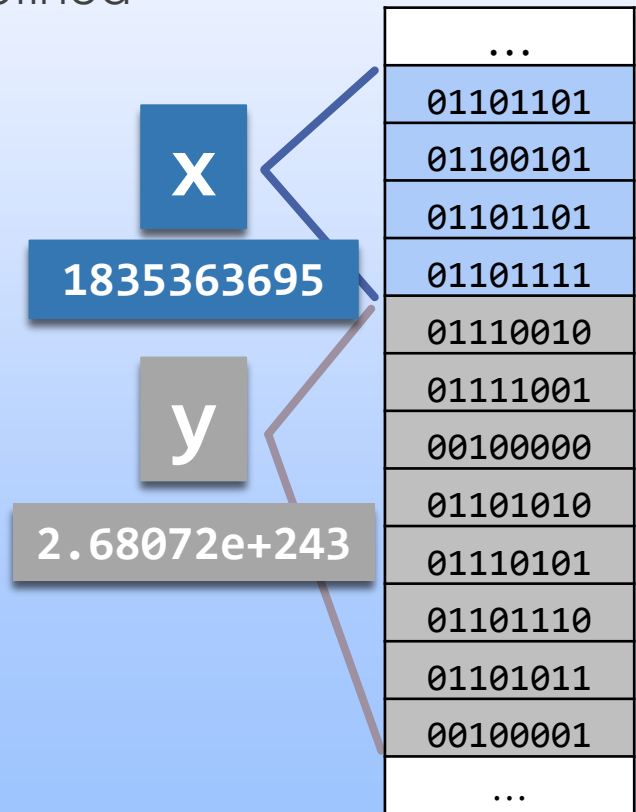
Uninitialized Objects

- What does default-initialization do?

For atomic types like `int`, `double`, `bool`, `char`, etc.
“do nothing” – the value is undefined

```
int main() {  
    int x;    // x = ??  
    double y; // y = ??  
}
```

- We often say the value of the object contains **memory junk**.
- Sometimes memory happens to be filled with 0s, but this is NOT guaranteed.



Arithmetic Operations

- C++ supports most common arithmetic operations.

	Operator	Example	Result
Addition	+	2 + 3	5
Subtraction	-	5 - 3	2
Multiplication	*	5 * 3	15
Exponentiation	None, use <code>std::pow()</code>		
Integer Division	/	11 / 4	2
Floating-point Division	/	11.0 / 4.0	2.75
Integer Modulo	%	11 % 4	3
Floating-point Modulo	None, use <code>std::modf()</code>		

Arithmetic Promotion

- ▶ The arithmetic operations require a common type.
- ▶ If numeric operand types differ, one operand may be **promoted** via implicit conversion.
- ▶ For example, consider the division operator:

```
1 / 2; // integer division
```

```
1.0 / 2; // double division, 2 is promoted to 2.0
```

```
1 / 2.0; // double division, 1 is promoted to 1.0
```

```
1.0 / 2.0; // double division
```

Integer Division and Modulo

➤ BOTH operands must be of type int

Expression	Result
0 / 3	0
1 / 3	0
2 / 3	0
3 / 3	1
4 / 3	1
5 / 3	1
6 / 3	2
7 / 3	2

Expression	Result
0 % 3	0
1 % 3	1
2 % 3	2
3 % 3	0
4 % 3	1
5 % 3	2
6 % 3	0
7 % 3	1

➤ Example:

```
int x = 153; // total seconds
int m = x / 60; // whole minutes
int s = x % 60; // leftover seconds
```

Exercise: Modular Arithmetic

- Say you have a Matrix of width 5 and height 3.

Each cell has an “index”.

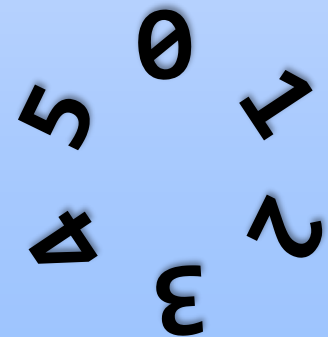
Use / and % to compute the row and column based on the index.

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14

```
int W = 5; int H = 3;
int index = 13;
int row = _____;
int col = _____;
```

- You're playing Uno with N players. Write code to update current to the next player and “wrap around” if needed.

```
int N; int current = ??;
current = _____;
```



Solution: Modular Arithmetic

- Say you have a Matrix of width 5 and height 3.

Each cell has an “index”.

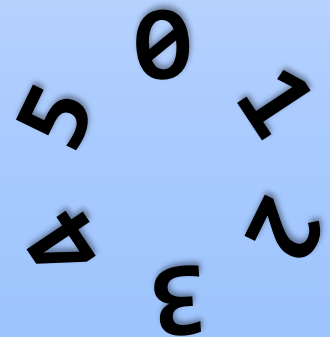
Use / and % to compute the row and column based on the index.

	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14

```
int W = 5; int H = 3;
int index = 13;
int row = index / W;
int col = index % W;
```

- You're playing Uno with N players. Write code to update current to the next player and “wrap around” if needed.

```
int N; int current = ??;
current = (current + 1) % N;
```



Relational Operations

All evaluate to
type bool.

- C++ supports the standard relational operations.

	Operator	Example	Result
Equality	<code>==</code>	<code>2 == 3</code>	<code>false</code>
Inequality	<code>!=</code>	<code>2 != 3</code>	<code>true</code>
Less	<code><</code>	<code>5 < 5</code>	<code>false</code>
Less or Equal	<code><=</code>	<code>5 <= 5</code>	<code>true</code>
Greater	<code>></code>	<code>'c' > 'a'</code>	<code>true</code>
Greater or Equal	<code>>=</code>	<code>0.1 + 0.2 >= 0.3</code>	<code>false</code>



Floating Point Precision

0.1 in IEEE754 64-bit binary representation

00111111 10111001 10011001 10011001
10011001 10011001 10011001 10011010

- There are infinitely many real numbers, but only a finite number (e.g. 2^{64}) of representable double values.

```
int main() {  
    cout.precision(20); // print up to 20 decimal places  
    cout << 0.1 << endl; // 0.10000000000000000555  
    cout << 0.2 << endl; // 0.20000000000000000111  
    cout << 0.3 << endl; // 0.29999999999999999889  
    cout << (0.1 + 0.2) << endl; // 0.30000000000000000441  
}
```

WARNING

- If you perform arithmetic on floating-point numbers, don't use `==` or `!=` to check the result.
- Instead, check if the numbers are close. See the **Comparisons** reference on project 1 for details.

Relational Operations

- ▶ “Chained comparisons” in C++ will not work as expected*:

NO

```
int x = 10;  
3 < x < 7; // (3 < x) < 7  
           // true  < 7  
           // 1    < 7  
           //      true
```

- ▶ Instead, use `&&` to test lower and upper bounds:

YES

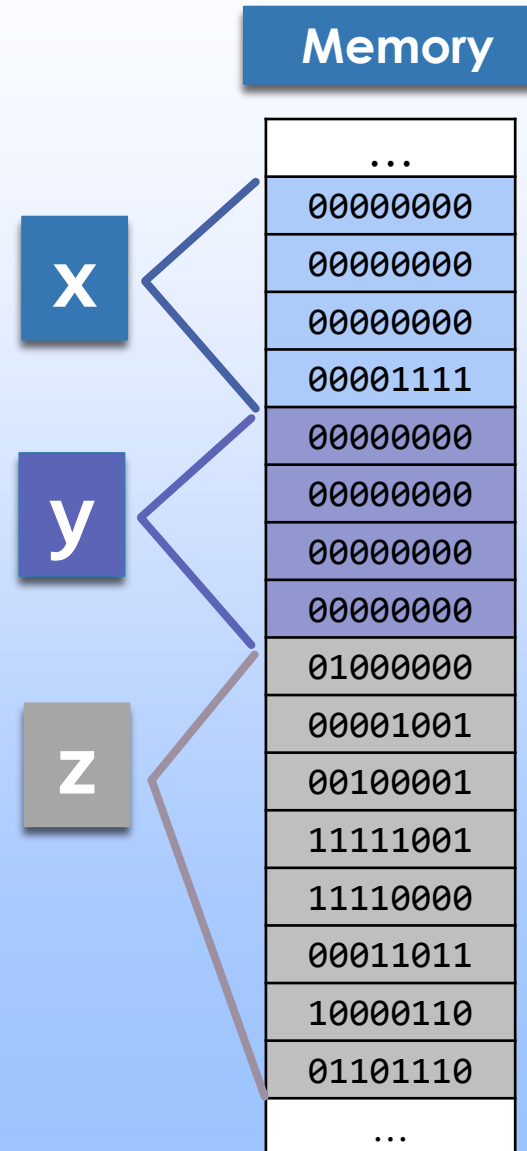
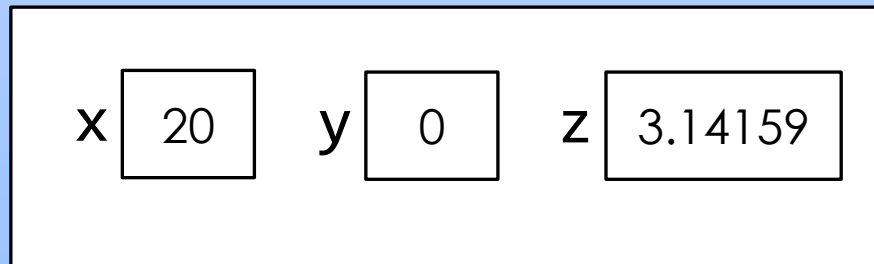
```
int x = 10;  
3 < x && x < 7; // (3 < x) && (x < 7)  
               // true  && false  
               //      false
```

*Most languages do not support this. Python is a notable exception.

Memory Diagrams

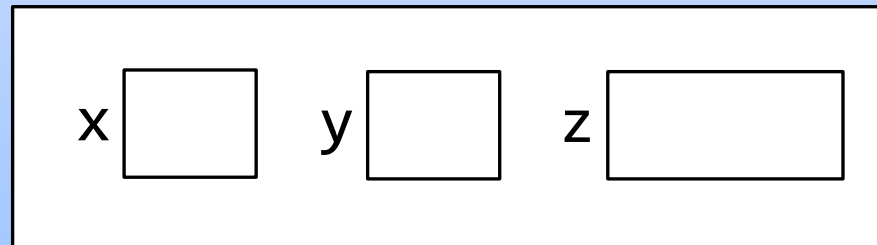
```
// Declare some variables  
int x = 10 + 5;  
int y = 0;  
double z = 3.14159;
```

- Simple memory diagrams are fine:
 - Specific binary encodings of data is beyond the scope of EECS 280.
 - The ordering of variables in memory is usually not significant.



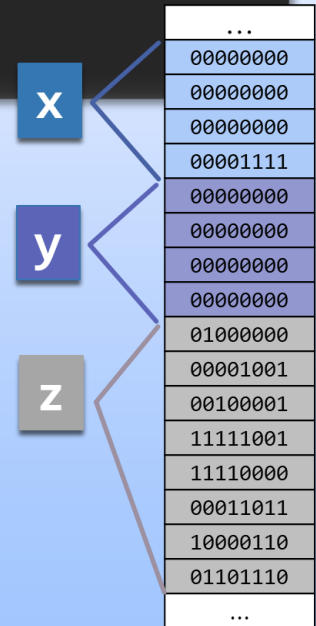
Assignment and Value Semantics

- C++ has **value semantics**.
- Variables always refer to the same object once they are declared.
- An **assignment** copies a *value* from one object to another.
 - It does NOT move a variable to refer to some other object.*
- Let's trace the assignments...



```
// Declare variables
int x = 10 + 5;
int y = 0;
double z = 3.14159;

// Assignments
y = x;
x = 20;
```



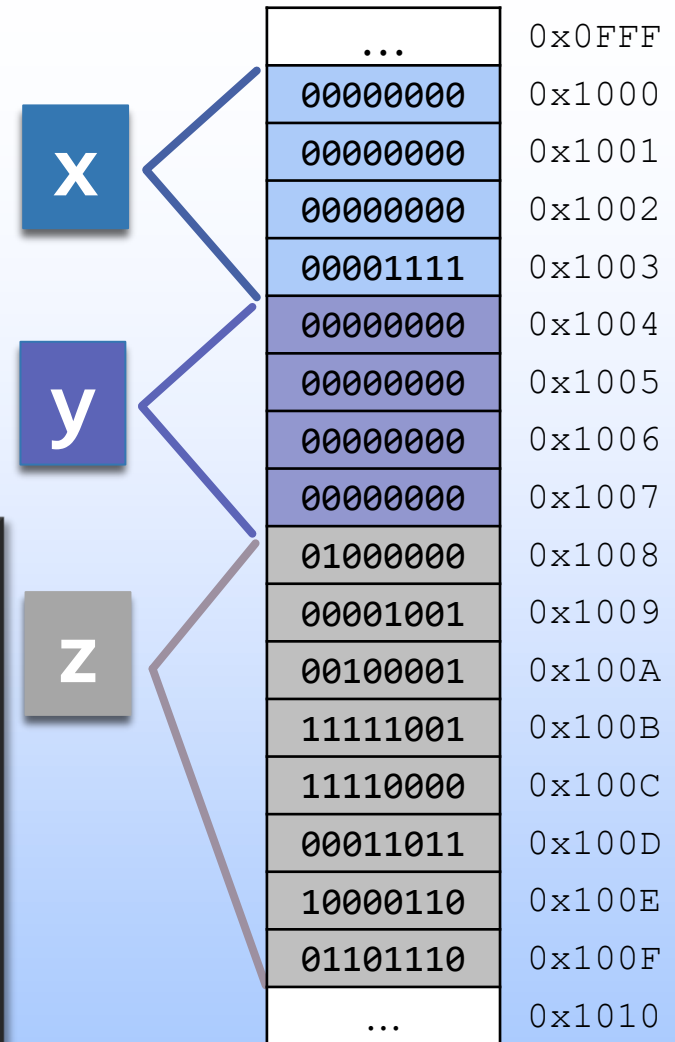
*Many languages (e.g. Java, Python, Javascript, etc.) primarily use **reference semantics**, where variables are indirect references to objects and assignment changes which object a variable refers to.

Addresses

- ▶ Every object lives exists at some **address*** in memory.
- ▶ The **& operator** yields the address of an object.

```
// Declare some variables
int x = 10 + 5;
int y = x;
double z = 3.14159;
int A;

cout << &x; // prints 0x1000
cout << &y; // prints 0x1004
cout << &z; // prints 0x1008
cout << &A; // prints 0x1010
```



Addresses are generally written in *hexadecimal* (i.e. base 16 with digits 0123456789ABCDEF).

*Generally, you don't get to pick the specific numeric address.

Value Semantics

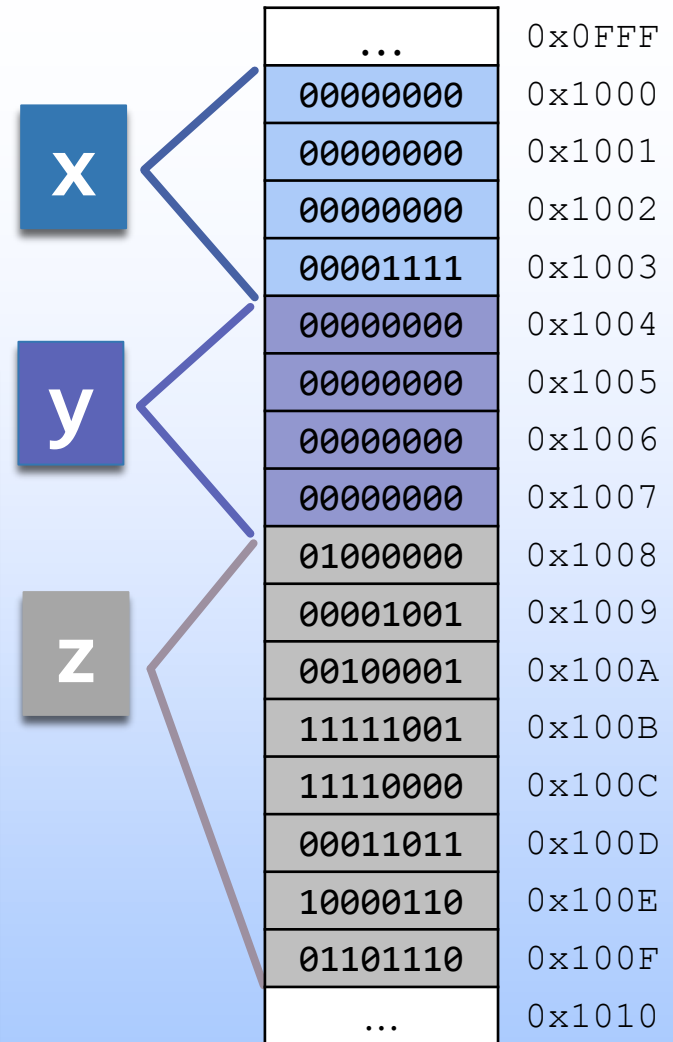
- Let's revisit **value semantics** by looking at **addresses**.

```
// Declare variables
int x = 10 + 5;
int y = 0;
double z = 3.14159;

cout << &x; // 0x1000
cout << &y; // 0x1004

// Assignments
y = x;
x = 20;

cout << &x; // still 0x1000
cout << &y; // still 0x1004
```



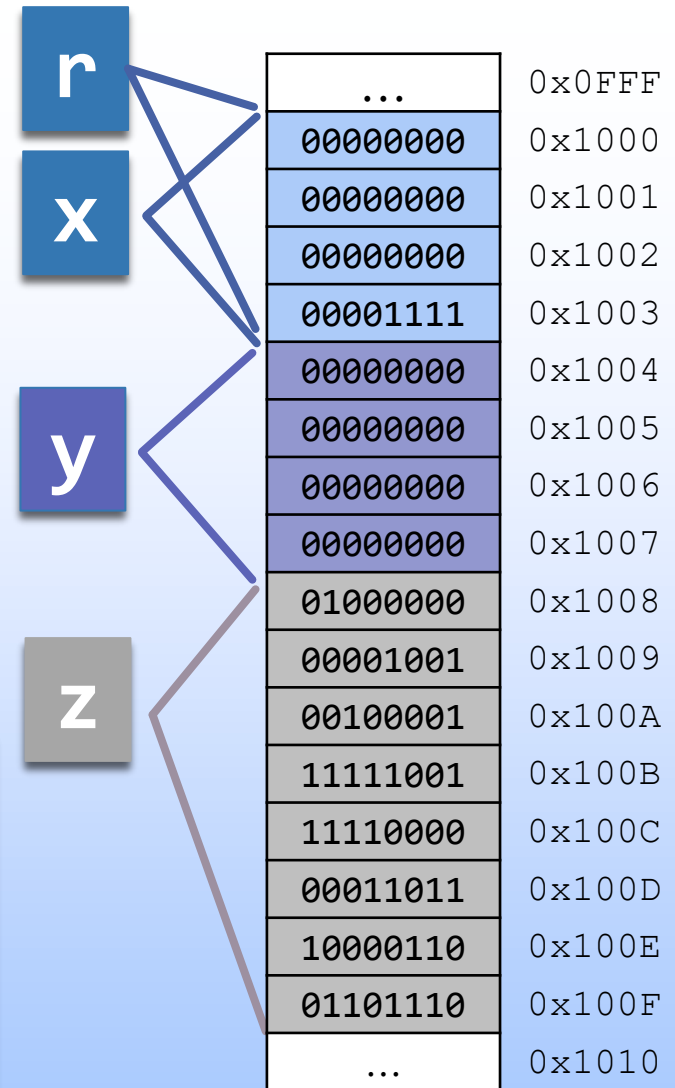
References

- A **reference** declaration binds a variable to an already existing object.
 - This gives an object **multiple names** or **aliases**.

`int &` = "reference to int"

```
// Declare variables
int x = 10 + 5;
int y = x;
int &r = x;
double z = 3.14159;

cout << &x; // 0x1000
cout << &y; // 0x1004
cout << &r; // 0x1000
```



r, x 0x1000
 y 0x1004
 z 0x1008

Example: References

- ▶ Let's trace this code...

```
// Declare variables
int x = 10 + 5;
int y = x;
int &a = x;
int &b = y;
a = 10;
cout << x << " " << a;
b = x;
x = a;
```

- ▶ Takeaways:
 - ▶ Changes to the reference “magically” affect the original.
 - ▶ After declaration, references still have value semantics.
 - ▶ You can't re-bind a reference.

We'll start again in five minutes.

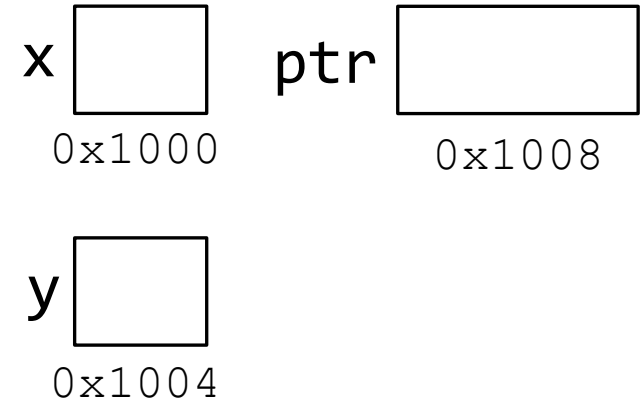


Pointers

- We can use addresses as **pointers** to indirectly access objects.
 - Example: a variable that stores the address of something else.

`int *` = “pointer to int”

- Use the **& operator** to get a pointer to an object.
- Use the *** operator** to **dereference** a pointer.
i.e. “follow the pointer”



```
int main() {
    int x = 3;
    int y = 5;
    int *ptr = &x;

    cout << ptr; // prints 0x1008
    cout << *ptr; // prints 3
    cout << &ptr; // prints 0x100c

    *ptr = 100; // set x to 100 via ptr

    ptr = &y; // re-point ptr at y

    *ptr = -42; // set y to -42 via ptr
}
```

& and * are Inverse Operations

& gives you the address of an object.

- “address of”
- Yields a pointer type. If you take the address of an `int`, you get an `int*`

*** gives you the object at some address.**

- “star”, “dereference”, or “indirection”
- Follows a pointer to find an object

Pointer Types

- There is a separate pointer type for each kind of thing you could point to, and you can't mix them.

```
int main() {  
    int x = 3;  
    double y = 4;  
    int * ptr1 = &x;  
    double * ptr2 = &y;  
    int ** ptr_ptr = &ptr1;  
}
```

References vs. Pointers

References	Pointers
An alias for an object	Stores address of an object

```
int main() {  
    int x = 3;  
    int &y = x;  
    int *z = &x;  
}
```

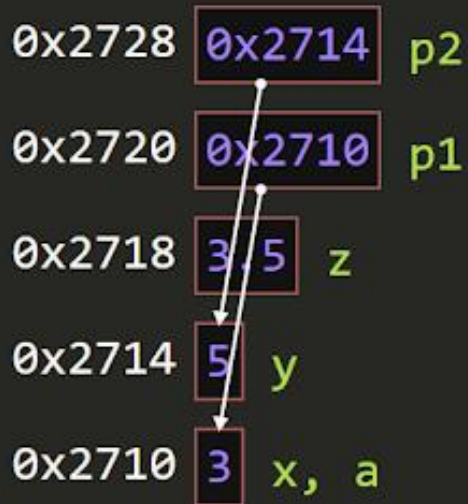
- You can change where a pointer points.
- You cannot re-bind a reference!

Exercise: Pointer Fundamentals

- Go to lobster.eecs.umich.edu
- Click the switch to turn on the “instant memory diagram”

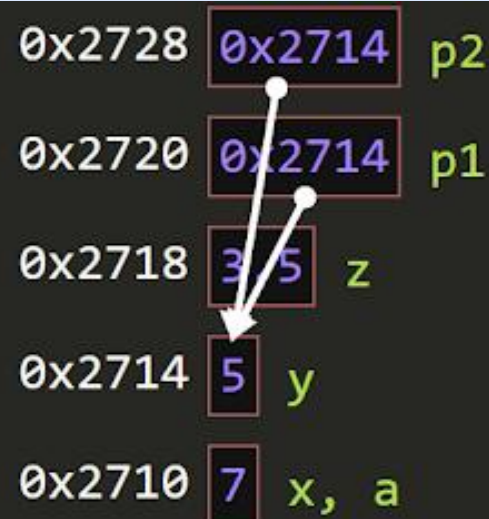
Task 1

Write code in `main()` to produce this state.



Task 2

Add two lines of code to change to this state, without using `x` or `y` directly.



Participation

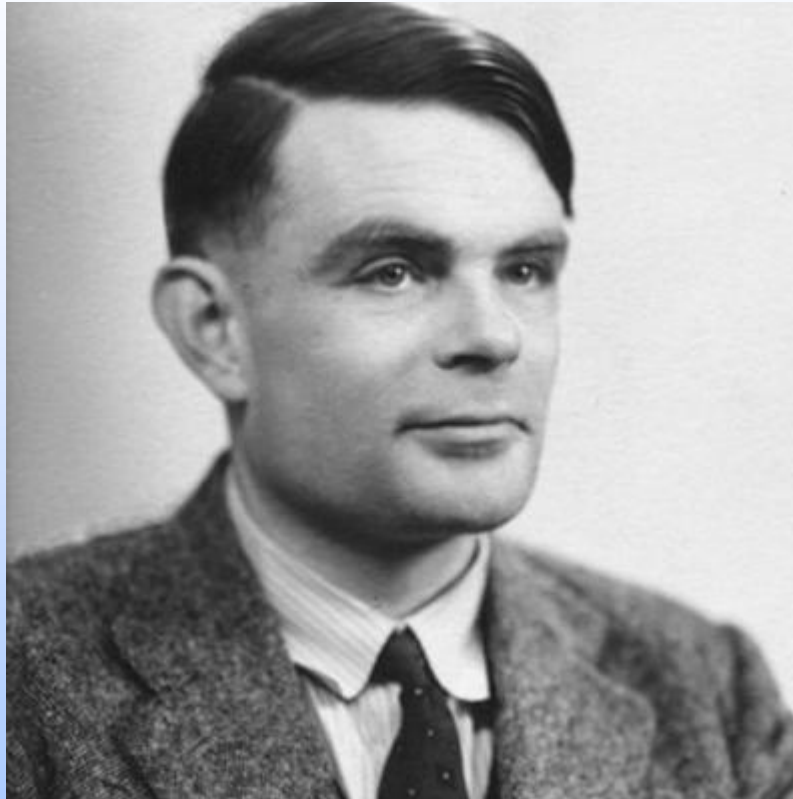
▶ bit.ly/41J659X







Alan Turing



Computer Scientist and Mathematician

Foundational work in theoretical computer science and artificial intelligence

Ada Lovelace

The First Computer Programmer



Diagram for the computation by the Engine of the N

Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data.					
						$1V_1$ ○ 0 1	$1V_2$ ○ 0 2	$1V_3$ ○ 0 4	$0V_4$ ○ 0 0	$0V_5$ ○ 0 0	$0V_6$ ○ 0 0
						1	2	n			
1	×	$1V_2 \times 1V_3$	$1V_4, 1V_5, 1V_6$	$\begin{cases} 1V_2 = 1V_2 \\ 1V_3 = 1V_3 \\ 1V_4 = 2V_4 \\ 1V_5 = 1V_5 \\ 1V_6 = 2V_6 \end{cases}$	$= 2n$	...	2	n	2n	2n	2n
2	-	$1V_4 - 1V_1$	$2V_4$	$\begin{cases} 1V_4 = 2V_4 \\ 1V_5 = 1V_5 \\ 1V_6 = 2V_6 \end{cases}$	$= 2n - 1$	1	...	...	$2n - 1$		
3	+	$1V_5 + 1V_1$	$2V_5$	$\begin{cases} 1V_5 = 2V_5 \\ 1V_6 = 1V_6 \end{cases}$	$= 2n + 1$	1	...	...	...	$2n + 1$	
4	+	$2V_5 + 2V_4$	$1V_{11}$	$\begin{cases} 2V_5 = 0V_5 \\ 2V_4 = 0V_4 \end{cases}$	$= 2n - 1$ $= 2n + 1$	...	...	...	0	0	...
5	+	$1V_{11} + 1V_2$	$2V_{11}$	$\begin{cases} 1V_{11} = 2V_{11} \\ 1V_2 = 1V_2 \end{cases}$	$= \frac{1}{2} \cdot 2n - 1$ $= \frac{1}{2} \cdot 2n + 1$	...	2	...	...	...	...
6	-	$0V_{13} - 2V_{11}$	$1V_{13}$	$\begin{cases} 2V_{11} = 0V_{11} \\ 0V_{13} = 1V_{13} \end{cases}$	$= -\frac{1}{2} \cdot 2n - 1 = A_0$	...	...	...	...	...	...
7	-	$1V_3 - 1V_1$	$1V_{10}$	$\begin{cases} 1V_3 = 1V_3 \\ 1V_1 = 1V_1 \end{cases}$	$= n - 1 (= 3)$	1	...	n	...	...	...
8	+	$1V_2 + 0V_2$	$1V_7$	$\begin{cases} 1V_2 = 1V_2 \\ 0V_7 = 1V_7 \end{cases}$	$= 2 + 0 = 2$	...	2	...	...	...	...
9	÷	$1V_6 \div 1V_7$	$3V_{11}$	$\begin{cases} 1V_6 = 1V_6 \\ 3V_{11} = 3V_{11} \end{cases}$	$= \frac{2n}{2} = A_1$	...	...	...	...	...	$2n$
10	×	$1V_{21} \times 3V_{11}$	$1V_{12}$	$\begin{cases} 1V_{21} = 1V_{21} \\ 3V_{11} = 3V_{11} \end{cases}$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...	...	...	...	...
11	+	$1V_{12} + 1V_{13}$	$2V_{13}$	$\begin{cases} 1V_{12} = 0V_{12} \\ 1V_{13} = 2V_{13} \end{cases}$	$= -\frac{1}{2} \cdot 2n - 1 + B_1 \cdot \frac{2n}{2}$	...	...	...	...	...	...
12	-	$1V_{10} - 1V_1$	$2V_{10}$	$\begin{cases} 1V_{10} = 2V_{10} \\ 1V_1 = 1V_1 \end{cases}$	$= n - 2 (= 2)$	1	...	...	...	...	...

A diagram by Lovelace contains the first recorded algorithm for implementation by a computer.

Exercise – Pointer Operations

```
int x = 1; // Assume x is allocated at address 0x2710
int y = 1; // Assume y is allocated at address 0x2714
int *ptr = &x; // Assume ptr is allocated at address 0x2718
```

```
x = 2;
*ptr = 3;

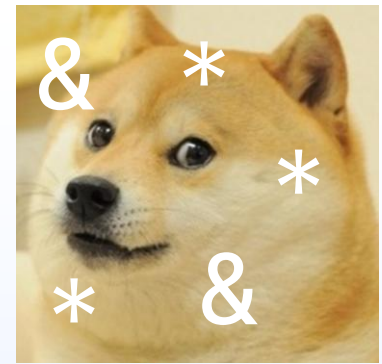
ptr = &y;
*ptr = 4;
```

Memory Diagram

► What is printed if each of the following are sent to cout?

x		ptr		&x		*&x	
y		*ptr		&ptr		&*ptr	

So Many * and &



➤ Used to specify a type...

➤ * means it's a pointer

```
int *ptr;
```

➤ & means it's a reference

```
int &ref;
```

➤ Used as an operator in an expression...

➤ * means get object at an address

```
cout << *ptr << endl;
```

➤ & means take address of an object

```
cout << &x << endl;
```

What can you do with pointers?

- ▶ Work with objects *indirectly*.
 - ▶ Implement reference semantics.
 - ▶ Use objects across different scopes.*
 - ▶ Enable subtype polymorphism.*
 - ▶ Keep track of objects in dynamic memory.*
 - ▶ Manipulate memory via low-level arrays.*
 - ▶ Build linked data structures.*
 - ▶ And more!

* We'll look at these later in the course.

Recall: Expressions

- Expressions may be composed of:

Literals

```
10
```

Variables

```
x cout
```

Function Calls

```
sqrt(x);
```

Operators

```
+ - >= << % !  
++ [ ] -> new
```

- Expressions **evaluate** to compute some result:

```
10 + x + sqrt(x);
```

- Expressions have a fixed **static type**.

```
int x = 10;      string str = "llama";  
  
x * 2; // value = ??? but always an int  
x < 5; // value = ??? but always a bool (i.e. true/false)  
str + str; // value = ??? but always a string
```

Expression Value Categories

- ▶ An expression's evaluated result has a **value category**¹:

lvalue

an object at some address in memory

`x` `*ptr` `v[2]`

rvalue

a value with no meaningful address

`5` `x + 2` `sqrt(2)` `&x`

- ▶ An lvalue may act as a rvalue by “looking up” its value.
- ▶ Relationship to assignment with =
 - ▶ lvalue – “left value”, could appear on left (or right) side
 - ▶ rvalue – “right value”, only allowed on the right side
- ▶ Relationship to address-of operator &
 - ▶ May only be applied to lvalues – why?

¹ C++ has a complex hierarchy of value categories. We present a simplified version.